

What is Modular Monolith Architecture

Overview

A modular monolith is a single application that is internally divided into well-defined modules. Each module represents a specific business responsibility and has clear boundaries. All modules are deployed together as one unit and usually share the same database. This architecture improves code organization and maintainability while avoiding the complexity of distributed systems.

When to use

A modular monolith is a good choice when the system is small to medium in size and the development team is limited. It works well when business requirements are still changing and fast development is needed. This architecture is suitable when the organization has limited DevOps skills or infrastructure, since it requires only one deployment pipeline and is easier to monitor and debug. It is also preferred when strong data consistency is required, because transactions across modules can be handled easily within a single database transaction. Performance-critical systems also benefit, as communication between modules happens in-process without network overhead.

Example Use Cases

- Internal enterprise systems such as HR or CRM applications
- Small to medium e-commerce platforms
- Content management systems
- Educational platforms with tightly integrated features

What is Microservice Architecture

Overview

Microservice architecture splits an application into many small, independent services. Each service is responsible for a specific business capability and can be developed, deployed, and scaled independently. Services communicate with each other through network protocols such as REST, gRPC, or message queues, and each service may have its own database.

When to use

Microservices are suitable for large and complex systems with high traffic and clear domain boundaries. This architecture is a good fit for organizations with multiple autonomous teams that need to work independently and deploy frequently. It is useful when different parts of the system have different scaling or performance requirements. Microservices are also preferred when fault isolation is important, as failures in one service are less likely to affect the entire system. They are well suited for cloud-native environments that use container orchestration and auto-scaling.

Example Use Cases

Large e-commerce platforms

Streaming services

Social media platforms

Financial trading systems

Large-scale IoT platforms